

Q.1. Write about inline function and explain it with example.

An inline function is a function in which the compiler replaces the function call with the actual function code during compilation. Instead of jumping to the function definition, the compiler directly inserts the code of the function at the place where it is called.

This eliminates the overhead of function calling and increases execution speed for small functions.

In C++, the keyword `inline` is used before the function definition to declare a function as inline.

Syntax of inline function:

```
inline return-type function-name(parameters)
{
    // function body
}
```

For example:

```
inline int square(int x) {
    return x * x;
}
```

Here, every time `square(n)` is called, the compiler will replace it with $n * n$.

Inline Function

```
inline int square(int x) {  
    return x * x;  
}  
  
int main() {  
    int a = square(5);  
}
```

Thus, no function call happens at runtime. Instead, the function body is directly substituted.

Advantages of Inline function

1. **Faster Execution** :- Removes overhead of function calls.
2. **Efficient for small function** :- Works best for simple, frequently used function.
3. **Improves Readability** :- Code remains modular and clean without affecting speed.
4. **Better Performance in loops** :- Inline functions are very effective when used inside loops.

Example :

```
inline int cube(int x) {  
    return x * x * x;  
}  
  
int main() {  
    for (int i = 1; i <= 5; i++) {  
        cout << "Cube of " << i << " = " << cube(i)  
            << endl;  
    }  
}
```

Here, each call of cube(i) will be replaced by $i * i$, avoiding repeated function calls.

Limitations of Inline Functions

1. Not suitable for large functions - increases code size (code bloat).
2. Excessive inlining can slow down performance because the executable size becomes large.
3. Compiler decision - The compiler may ignore the inline request if it finds the function too complex.
4. Not recommended for recursion - Recursive functions are not generally made inline.

Inline function to calculate maximum of two number.

```
#include <iostream>
using namespace std;
```

```
inline int maximum(int a, int b){
    return (a > b) ? a : b;
}
```

```
int main() {
    int x = 10, y = 25;
    cout << "Maximum is : " << maximum(x, y);
}
```

Output :

Maximum is : 25

Key Points

- Use inline keyword only for small functions.
- The compiler may or may not inline function, depending on optimization.
- Inline functions are mainly used to reduce function call overhead.
- Recursive and complex functions are generally not inlined.

Q.2. Write a program of friend function.

A friend function is a special function that is not a member of a class, but it is given special permission to access the private and protected data of that class.

Syntax:

```
class ClassName {  
    private:  
        int data;  
    public:  
        friend void functionName(ClassName obj);  
};
```

Here, `FunctionName` is a non-member function but still can access private data of `ClassName`.

* Program of Friend Function.

```
#include <iostream>  
using namespace std;
```

```
class Box {  
    private:  
        int length;    // private data member  
    public:  
        // constructor to initialize length
```

```
Box (int l) {  
    length = l;  
}
```

// declaring friend function.

```
friend void printlength (Box b);  
};
```

```
void printlength (Box b) {
```

// can access private data of Box

```
    cout << "length of the box is : " << b.length << endl;  
}
```

```
int main() {
```

```
    Box b1(25); // create object with length = 25
```

```
    printlength (b1); // call of friend function.
```

```
    return 0 ;
```

```
}
```

Output:

length of the box is : 25

- length is private, so normally it can't be accessed outside the class.
- The friend function is not a member of the class, but it can access private data because of the friend keyword.

A.3. What is Virtual function and Pure virtual function?

Virtual Function:-

A virtual function is a member function in a base class that is declared using the keyword `virtual`. It allows derived classes to override it, and when the function is called through a base class pointer or reference, the derived class version is executed at runtime.

Syntax

```
class Base {  
    public:  
        virtual void display() {  
            cout << "Display from Base class" << endl;  
        }  
};  
class Derived : public Base {  
    public:  
        void display() override {  
            cout << "Display from Derived class" << endl;  
        }  
};
```

Key Characteristics of Virtual Functions.

1. Declared using the `virtual` keyword inside the base class.

2. Enables runtime polymorphism (late binding).
3. Must be a member function of a class.
4. If not overridden in the derived class, the base class version is invoked.
5. Can be accessed using base class pointers or references.
6. They consume extra memory due to vtable mechanism.
7. Constructors can't be virtual, but destructors can and often should be virtual.

Advantages of Virtual Functions

1. Runtime Polymorphism :- Correct function is chosen at runtime depending on object type.
2. Code Reusability :- Base class pointers can handle different derived objects.
3. Flexibility :- Makes class hierarchies extensible without changing existing code.
4. Clean Design :- Promotes separation of general behavior (base class) and specific behavior (derived class).

Disadvantages of Virtual Functions

1. Slight Performance Overhead :- Because of run-time resolution using vtable.
2. Increased Memory Usage :- Each class with virtual functions maintains a vtable.
3. Complexity :- Improper use may lead to confusing code and bugs.

Pure Virtual Function.

A pure virtual function is a virtual function that has no definition in the base class and is meant to be overridden by derived classes.

It is declared by assigning $= 0$ in the base class.

Such a class is ~~also~~ called an abstract class, and objects of abstract classes can't be created. They serve as blueprints for derived classes.

Syntax

```
class Base {  
    public :
```

```
virtual void show() = 0;  
};
```

Key Point about Pure Virtual Functions

1. Declared with $= 0$ in the base class.
2. Makes the class abstract.
3. Derived classes must override pure virtual functions.
4. Abstract classes can't be instantiated directly, but pointers to them can exist.
5. They are used to define interfaces in C++.

Virtual functions and pure virtual functions are the backbone of runtime polymorphism in C++.

- A virtual function provides flexibility by allowing derived classes to redefine functions while still allowing base class pointers to call them at runtime.
- A pure virtual function goes a step further by enforcing that derived classes must implement the function, making the base class abstract.

Q.4. Explain Inheritance with its type.

Inheritance is a fundamental concept of object - Oriented Programming (OOP) that allows a new class (derived class / child class) to acquire the properties and behaviors (data members and member function) of an existing class (base class / parent class).

It promotes reusability, extensibility, and code organization.

Inheritance means creating a new class using an already existing class.

Why is Inheritance used?

1. Code Reusability - Reuse methods and attributes of an existing class.
2. Extensibility - Extend existing classes with new features.
3. Maintainability - Easy to update code since changes in base class reflect in derived class.
4. Polymorphism Support - Helps in method overriding and dynamic binding.

Types of Inheritance.

1. Single Inheritance:

In single inheritance, a class derives from only one base class.

Example:-

```
class Animal {  
    public:  
        void eat() {  
            cout << "Eating ---- " << endl;  
        }  
};  
class Dog : Public Animal {  
    public:  
        void bark() {  
            cout << "Barking ---- " << endl;  
        }  
};
```

2. Multiple Inheritance

In multiple inheritance, a class can inherit from more than one base class.

Example:-

```
class Animal {  
    public:  
        void eat() { cout << "Eating ---- " << endl; }
```



```

};
class Bird {
    public:
        void fly() { cout << "Flying---" << endl; }
};
class Bat : public Animal, public Bird {
    public:
        void sleep() { cout << "Sleeping---" << endl; }
};

```

3. Multilevel Inheritance

In this type, a class derives from another class, which is also derived from another class. (A chain of inheritance).

Example :

```

class LivingThing {
    public:
        void breathe() { cout << "Breathing---" << endl; }
};
class Animal : public LivingThing {
    public:
        void eat() { cout << "Eating...." << endl; }
};
class Dog : public Animal {
    public:
        void bark() { cout << "Barking...." << endl; }
};

```

4. Hierarchical Inheritance

In this type, multiple classes inherit from the same base class.

Example:

```
class Animal {  
    public:  
    void eat() { cout << "Eating ----" << endl; }  
};  
class Dog : public Animal {  
    public:  
    void bark() { cout << "Barking --" << endl; }  
};  
class Cat : public Animal {  
    public:  
    void meow() { cout << "Meowing ----" << endl; }  
};
```

5. Hybrid Inheritance

A combination of more than one type of inheritance (e.g., Multiple + multilevel). It often leads to the Diamond Problem in C++.

Example:

```
class A {  
    public:  
    void display() { cout << "Class A" << endl; }  
};
```



```
class B : public A { } ;  
class C : public A { } ;  
class D : public B , public C { } ; //diamond Problem.
```

6. Hybrid / Dominant in Python (Extra Case).

Python supports multiple inheritance more smoothly by using Method Resolution Order (MRO) to solve conflicts.

Inheritance allows us to reuse and extend classes. The five main types are Single, Multiple, Multilevel, Hierarchical and Hybrid.

Access specifier can be public, protected or private.

- Public Inheritance :- Base class members of keeps their original access level.
- Protected Inheritance :- Public members of base become protected in derived.
- Private Inheritance :- Public and protected members of base become private in derived.

Q.5. What are stream classes in C++.

In C++, stream means a flow of data. Input and output (I/O) in C++ are managed with the help of stream classes. A stream is simply a sequence of bytes flowing from a source to a destination.

- **Input Stream** - Data flows into the program (e.g., - Keyboard, file).
- **Output Stream** - Data flows out of the program (e.g., screen, file).

To handle these flows in a systematic way, C++ provides stream classes which are defined in the standard libraries `<iostream>`, `<fstream>`, and `<sstream>`.

In C++, streams are represented by a hierarchy of classes, known as stream classes, defined inside the header. These classes are part of the Standard Template Library (STL) and provide powerful and flexible ways to handle data.

Importance of Stream Classes.

Before C++, in C language, input-output was handled using functions like `scanf()`, `printf()`, `fopen()`, `fread()`, etc. These functions

were not type-safe and required format specifiers, which often led to errors.

C++ introduced stream classes which:

1. Provide type safety (no need of format specifiers).
2. Make the code easier to read and maintain.
3. Support operator overloading (>> for input, << for output).
4. Allow manipulators (endl, setw, etc..) to format the output.
5. Offer a hierarchical class structure, so the same functions can be used for console and file I/O.

Stream Class Hierarchy in C++

The stream classes in C++ are organized in a hierarchy. The main classes are defined in `<iostream>` and `<fstream>` libraries.

A. Core Classes.

- `ios` → Base class for all stream classes. It provides basic formatting and error-handling facilities.

- `istream` → Derived from `ios`. Handles input operations (like `cin`).
- `ostream` → Derived from `ios`. Handling output operations (like `cout`).
- `iostream` → Derived from both `istream` and `ostream`. Supports both input and output.

B. File Stream Classes.

Defined in `<fstream>` header.

- `ifstream` → Derived from `istream`. Used for file input (reading).
- `ofstream` → Derived from `ostream`. Used for file output (writing).
- `fstream` → Derived from `iostream`. Used for both file input and output.

C. String Stream Classes

Defined in `<sstream>` header.

- `istringstream` → Derived from `istream`. Reads data from strings.

- `ostringstream` $\rightarrow$ Derived from `ostream`. Writes data into strings.
- `stringstream` $\rightarrow$ Derived from `iostream`. Supports both input and output with strings.

* Features of stream classes

1. Operator Overloading

`>>` is overloaded for input.
`<<` is overloaded for output.

Example :

```
int x;  
cin >> x;  
cout << x;
```

2. Type Safety

No format specifiers are required, unlike `printf` and `scanf`.

3. Manipulators :

Formatting I/O using manipulators like `endl`, `setw`, `setprecision`.

4. Error Handling

Stream classes provides function like `eof()`, `fail()`, `bad()`, `good()` to check the state of the stream.

* Advantages of Stream Classes.

1. Unified approach for console, file and string I/O.
2. Strongly type-checked and safer than C-style I/O.
3. Easy to use because of operator overloading.
4. Highly flexible due to manipulators and formatting options.
5. Portable across different systems.

* Limitations

- More complex than C-style I/O for beginners.
- Slightly slower than traditional C functions due to additional overhead.
- Requires understanding of class hierarchy.